DEPARTMENT OF SOFTWARE ENGINEERING
Faculty of Computing & Information Technology (FCIT)
University of the Punjab, Lahore – PAKISTAN



MedBill D1 Agile Report

Product Backlog and Definition of Done

BSEF23-51 MedBill

Bachelors of Science in Software Engineering (2023-2027)

By

Hasan Abdullah BSEF23M552

Irfan Saleem BSEF23M553

Supervised by:

Mr Umair Babar
Primary Supervisor

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY, UNIVERSITY OF THE PUNJAB, LAHORE.



Contents

1. Product Overview....2
2. System Design High Level....2
3. Features and Scope....3
4. Product Backlog....4
5. Prototype Demonstration....5
6. Progress Snapshot....7
7. Definition of Done....7
8. Development Approach....8
9. CI CD Plan Brief....8
10. Next Steps Semester 8....9

1. Product Overview

MedBill is a medical billing project for small clinics and billing teams. Our current D1 increment demonstrates the PDF verification part: upload or select documents, extract their text, check claim fields, obtain text-only AI assistance, and route the original PDFs to clear output folders.

Problem. Reviewing claim PDFs manually makes missing identifiers, unclear codes and incomplete provider details easy to overlook. Billing teams need an inspectable document-processing flow before adding a complete patient-to-payment system.

Target users. Billing staff and supervisors review synthetic claim documents in this public, login-free demonstration. Real patient records must not be uploaded. Authentication and accountable roles remain required for a private operational release.

D1 solution. We provide one working upload-to-results flow, plus a step-by-step debug mode. PostgreSQL stores metadata and reports; R2 stores PDFs and backups. Accepted means the current technical checks passed, not that an insurer approved the claim.

2. System Design High Level

One Next.js application serves both the React interface and Node.js API routes on Vercel. The backend reads PDFs from R2, performs extraction or Tesseract OCR, applies deterministic checks, calls Gemini with extracted text, and moves the original PDF. Ollama Cloud is a fallback. Prisma persists document and debug state in Supabase PostgreSQL.

Next.js browser UI	Next.js Node.js API	Persistent services
Upload, select, run, debug, inspect	Text/OCR, checks, AI, routing	Prisma/PostgreSQL metadata; R2 PDFs

Technology choices

- Frontend: Next.js App Router, React and TypeScript with one file-selection area and a prominent run action.
- Backend: Next.js Node.js Route Handlers; embedded-text extraction and bundled Tesseract.js OCR. No separate Python server.
- Database: Supabase PostgreSQL through Prisma ORM and pooled connections. Supabase Auth, Storage and Data API are not used.
- Files and AI: private Cloudflare R2 input/output/backups; Gemini primary and Ollama Cloud fallback, receiving text only.
- Deployment and checks: Vercel Hobby, lint/build, API smoke checks and browser-driven production verification.

3. Features and Scope

Core epics

- PDF intake and selection of previously uploaded files
- Text extraction and local OCR
- Explainable claim field checks and document routing
- Text-only AI extraction with provider fallback
- Complete runs and inspectable debug checkpoints
- Original PDF and Markdown report downloads
- R2 backup and reset with restoration to input

D1 implementation boundary

Implemented	Partial or limited	Remaining proposal work
PDF upload and old-file selection	4 MB and 50-page limits	Patient/insurance CRUD and roles
Embedded extraction and Node OCR	OCR can misread fields	Measured dataset evaluation
Field checks and four output folders	Syntax and presence only	Insurer eligibility and code lookup
Gemini text extraction; Ollama fallback	Quota-dependent; local fallback is labelled	Trained denial-risk model
Debug checkpoints and downloads	Public synthetic-data demo	CMS-1500 claim generation
Backup/reset and Vercel deployment	Hosted CI and Git integration not enabled	Payments, denials and financial reports

The original proposal also includes patient records, charge entry, CMS-1500 generation, payments, denials, financial reports and accountable roles. Those modules are not implemented in this version. We request supervisor confirmation of this narrower D1 increment and the revised Next.js/PostgreSQL stack.

4. Product Backlog

Must stories below define the demonstrated PDF increment. Broader billing and security stories remain in the backlog; Done refers only to the stated acceptance check.

ID	Priority	User story and acceptance check	D1 status
DOC-01	Must	As a reviewer, I upload a valid PDF and find it in input; invalid content is refused.	Done
DOC-02	Must	As a reviewer, I select any input batch; unselected files remain unchanged.	Done
TXT-01	Must	As a reviewer, I obtain embedded text or OCR text before AI; OCR uses no AI calls.	Done
VAL-01	Must	As a reviewer, I see checks for member ID, code format, amount, date and provider.	Done
AI-01	Must	As a reviewer, I inspect text-only extraction and the actual provider used.	Done
ROUTE-01	Must	As a reviewer, I find originals in accepted, review, rejected or failed folders.	Done
DBG-01	Should	As a reviewer, I run all five debug stages separately and see returned data.	Done
OPS-01	Must	As a reviewer, I back up/reset; output clears and all PDFs return to input.	Done
AUTH-01	Must later	As an operational user, I access private records through accountable roles.	Planned
BILL-01	Must later	As a biller, I manage patients/charges and create CMS-1500 claims.	Planned
FIN-01	Must later	As a biller, I post payments/denials and reconcile financial reports.	Planned
ML-01	Must later	As the team, we evaluate a denial-risk model against a baseline on held-out data.	Planned

5. Prototype Demonstration

Open https://medbill-steel.vercel.app without login. Select one or more existing input PDFs, then choose Complete flow or Debug step-by-step. Show extraction, checks, AI output and the final folder. Inspect a result and download its original PDF/report. Demonstrate backup/reset only after explaining that it restores every PDF to input and clears output.

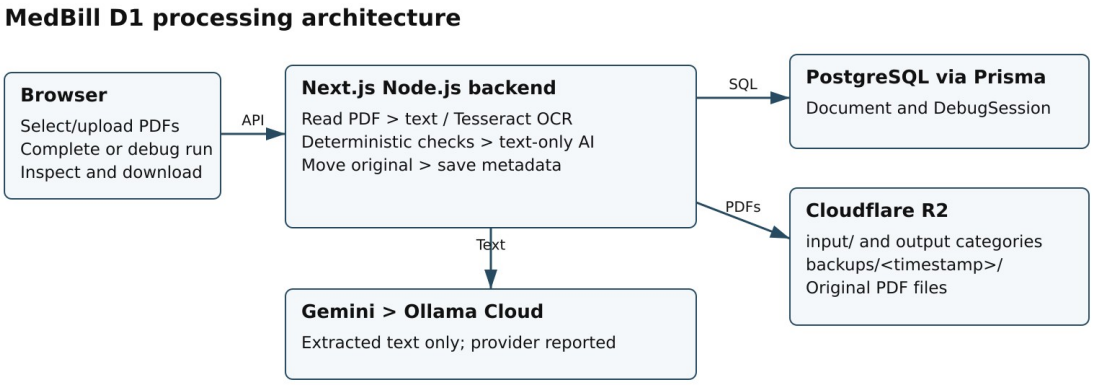


Figure 1 Current Next.js processing architecture

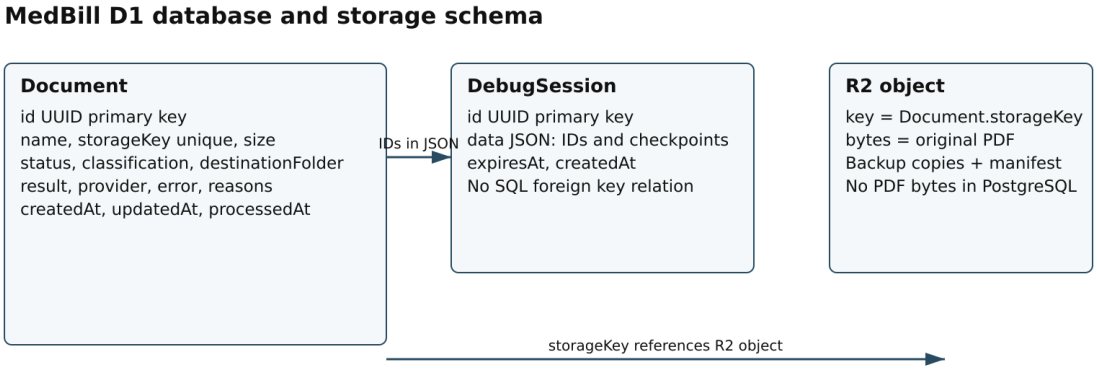


Figure 2 Current Prisma entities and their document references

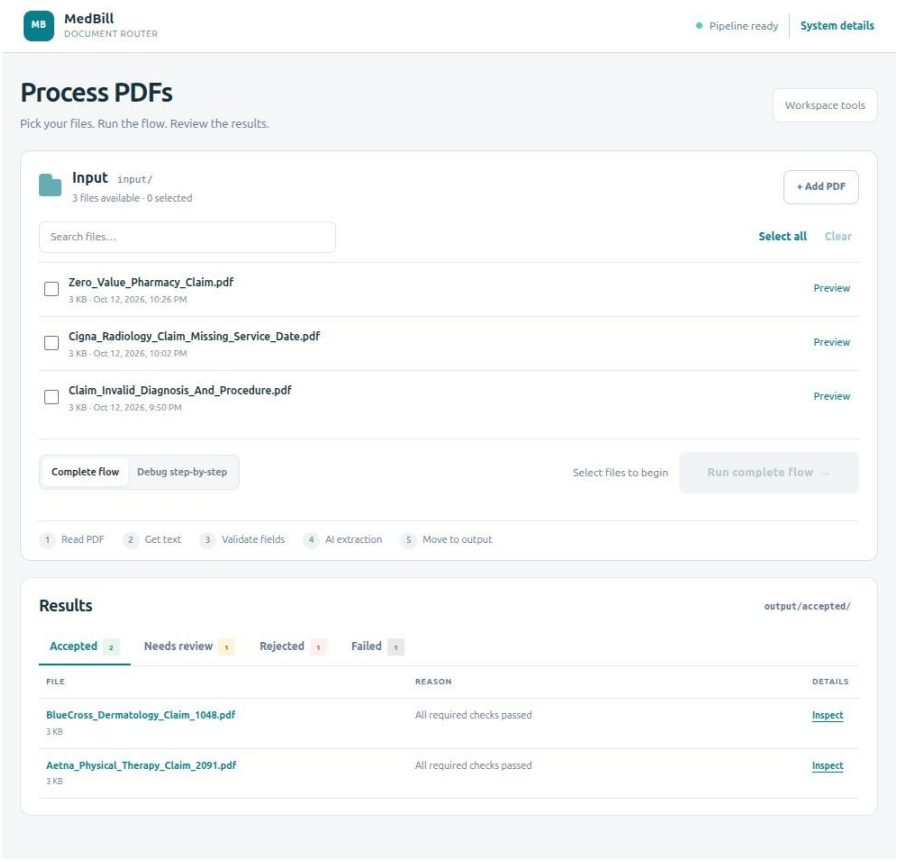


Figure 3 Deployed dashboard after production verification

Current limitations

- The public demo accepts only synthetic PDFs, with a 4 MB upload cap and a 50-page extraction cap. Free hosting and AI quotas remain finite.
- Checks identify field presence and code format; they do not verify insurer eligibility, code-set validity or claim reimbursement. Uncertain OCR goes to Needs review.
- Cloud language-model extraction is integrated, but a trained and evaluated denial-risk model is not. Patient CRUD, CMS-1500 creation, payments and financial reporting remain future work.

6. Progress Snapshot

Live app: https://medbill-steel.vercel.app. GitHub: https://github.com/hasanabdullah919/medbill_bsef23-51. D1 source snapshot: commit 3961a53; deployed runtime: f36bdcc. Production verification was completed on 13 October 2026.

Evidence	Result	Meaning
Vercel health and services	Passed	Real PostgreSQL and R2 connectivity
Production API smoke checks	Passed	Upload, guards, routing, PDF/report downloads and debug steps
Browser batch runs	Passed	Two-file debug and three-file complete run; all output categories
Forced OCR on Vercel	94% confidence; 0 AI calls	One synthetic page; engine estimate, not general accuracy
Backup/reset and physical R2 listing	Passed	Output emptied, files restored, backup preserved
Build, lint and production audit	Passed; 0 runtime advisories	Five high development-tool advisories remain

The evidence folder includes current diagrams, the dashboard screenshot and the recorded production checks. Eight distinct synthetic demo PDFs are included. Source code is frozen separately without credentials, dependencies, build output or the earlier Django prototype.

7. Definition of Done

A story is Done when its stated UI, API, storage and database behavior has been observed with synthetic data. Planned review and CI policies below are not claimed as completed activities.

- The story has an observable acceptance check and a backlog ID.
- Validation and file routing run on the backend; future restricted features require backend authorization.
- The UI, API, PostgreSQL and R2 paths work together with synthetic data.
- Happy, invalid-input and failure cases have recorded checks; repeatable API tests are included.
- The Next.js production build succeeds and the deployed health endpoint checks both services.
- No API keys, passwords, private records, environment files or generated dependencies enter the submission.
- A team review and supervisor scope confirmation are required before calling a broader release approved.
- The report, backlog, source snapshot and screenshots describe the same demonstrated increment.

8. Development Approach

The proposal uses two-week Scrum sprints, with planning, review and retrospective. This increment proves a complete document-processing path. Future sprints should add tested billing features rather than disconnected screens. Sprint ceremony records and completed peer reviews are not supplied as evidence.

Branching strategy

- main currently contains the published application and documentation commits.
- A develop branch is planned for integrated sprint work; this package does not claim it is enforced.
- Feature/fix branches, pull-request review and protected checks are the proposed team workflow.

Change workflow

Planned workflow: refine a story, implement a vertical slice, run checks, obtain peer review, demonstrate it and update the report. Current deployment uses Vercel CLI; automatic GitHub deployment and a hosted CI workflow are not enabled.

Team responsibilities

- Hasan Abdullah: proposed Scrum Master and backend/AI lead, as listed in the proposal.
- Irfan Saleem: proposed Product Owner and frontend/database lead, as listed in the proposal.
- Both students should be able to explain setup, rules, limitations, tests and the live demonstration.

9. CI CD Plan Brief

Current checks are executed before manual deployment. Hosted GitHub Actions and protected-branch gates are planned. Vercel builds the Next.js app; successful compilation alone is insufficient, so real production endpoints and file movement are also checked.

Stage	Check	Current state or planned gate
Local code	npm run lint; npm run build	Executed; required before deployment
Database	Prisma generation and migrations	Applied to PostgreSQL
Live system	API smoke checks, browser demo, R2 folder listing	Executed on production
Hosted CI	GitHub Actions, peer review and protected branches	Planned; block promotion once configured

Keep credentials in server-side environment variables, apply Prisma migrations before first use, deploy with Vercel CLI and verify /api/health plus a synthetic run. Repeat backup/reset checks on a test workspace. Free-tier quotas and public-demo privacy restrictions remain operational constraints.

10. Next Steps Semester 8

Target	Planned outcome
Scope review	Confirm the D1 PDF increment and stack changes against the proposal.
Sprint 1	Add authentication, roles, patient/insurance management and accountable audit events.
Sprint 2	Implement charges, permitted code lookup, claim states and CMS-1500 PDF generation.
Sprint 3	Add payment, adjustment, denial and outstanding-balance logic with fixtures.
Sprint 4	Build reconciled financial reports and an approved, measured ML experiment.
Release	Configure CI/review gates, privacy controls, regression/UAT, monitoring and documentation.

For D1, demonstrate the working PDF increment and ask the supervisor to confirm its scope against the original billing proposal. Do not present output Accepted as payer acceptance, OCR confidence as accuracy, or cloud extraction as a trained denial-risk model.